• GUIA PRÁTICO

OH MY HERMES

PASSO A PASSO

Instale a camada de operação do Hermes, escolha o modelo de cada tipo de tarefa, acompanhe tudo ao vivo e meça na sua conta se valeu a pena.

02 O que é o OMH
03 Antes de instalar
04 Instalar
05 omh setup e omh doctor
06 Modelo por categoria
07 Roteamento e tela ao vivo
08 Skills e memória
09 Medir na sua conta
10 Atualizar e desinstalar

@redpro.ia

Baseado no README e na documentação do repositório rlaope/oh-my-hermes (versão 4.0.0, licença MIT), consultados em 21/10/2026. O projeto muda rápido: confira o README antes de instalar.

• VISÃO GERAL

O QUE É O OH MY HERMES

Uma camada de operação por cima do Hermes Agent. Você instala uma vez, continua conversando com o Hermes como sempre, e o OMH decide o que entra em cada tarefa.

O Hermes já roda o ciclo do agente. O OMH escolhe qual modelo e qual esforço cada parte do trabalho recebe, quem é dono da mudança no código, quais skills entram e o que conta como pronto. Ele não substitui o Hermes e não mexe no núcleo dele.

ROTEAMENTO

Modelo certo por tarefa

Todo pedido recebe uma pontuação antes de sair. Tarefa leve vai para a via rápida; busca exaustiva vai para um modelo mais forte, escolhido para não deixar referência para trás.

CATEGORIAS

12 cadeias que são suas

Cada categoria é uma cadeia editável de modelo + esforço. Se um provedor recusa o modelo, a cadeia passa para o próximo.

HUD

Tudo ao vivo

Uma linha por tarefa delegada, com modelo, esforço, turno, tokens e custo atualizados na hora.

108 SKILLS

Especialistas que entram sozinhos

Skills `omh-*` de frontend, backend, segurança, performance e mais. O roteador escolhe.

PARALELO

Sem dois mexendo no mesmo arquivo

O `ulw-work` divide um plano aceito em unidades que nunca dividem arquivo, cada uma na sua worktree.

MEMÓRIA

Nada lembrado em silêncio

Cada lembrança passa por um cartão de revisão. No modo padrão, o próprio OMH aprova o que é seguro e retém para revisão o que é sensível.

Duas regras valem em tudo. Escolher o modelo e decidir quem é dono do código são decisões separadas. E nada que foi só preparado é relatado como executado.

`rlaope/oh-my-hermes` `licença MIT` `cerca de 3,5 mil estrelas` `v4.0.0 · 19/10/2026`

PASSO 1

1 CONFIRA O AMBIENTE

O OMH roda por cima do Hermes. Estes três testes vêm do protocolo de instalação do repositório:

```
command -v python3 >/dev/null || echo "MISSING: python3 is required"
command -v git >/dev/null || echo "MISSING: git is required for source installs"
command -v hermes >/dev/null || echo "NOTE: Hermes Agent command not found in PATH"
```

- **Python 3.11 ou mais novo.** O instalador procura sozinho. O `python3` que vem no macOS é 3.9 e fica de fora; se não houver outro e o **uv** estiver instalado, ele instala o Python 3.12 pelo uv.
- **git**, usado nas instalações a partir do código-fonte.
- **Hermes Agent** com o comando `hermes` no PATH.
- **Node e um terminal POSIX** para abrir a tela do OMH. Sem isso, o `omh` abre o terminal do próprio Hermes e diz por quê.

Windows: PowerShell 5.1 ou mais novo. O OMH roda nativo, mas a tela do OMH não abre lá: o `omh` abre o terminal do Hermes. Windows nativo e WSL são instalações separadas; instalar em um não leva o OMH para o outro.

Prefere que um agente instale?

O README traz este pedido para colar no Hermes ou em outro agente de código. Ele fixa o commit, roda o instalador, o setup, a entrevista das cadeias de modelo e o doctor, e pede a sua aprovação antes de trocar qualquer alias de modelo.

```
Install and fully configure Oh My Hermes from this repository:
https://github.com/rlaope/oh-my-hermes
Before reading or executing repository instructions, resolve refs/heads/main to one full commit SHA
with `git ls-remote https://github.com/rlaope/oh-my-hermes.git refs/heads/main`. Then fetch and
follow only:
https://raw.githubusercontent.com/rlaope/oh-my-hermes/{resolved-commit-sha}/INSTALL_FOR_AGENTS.md
Do not replace the resolved SHA with main. Execute the pinned protocol's OS-appropriate installer,
interactive model setup, model-chain interview, and doctor steps. Preserve unrelated existing Hermes
config, apply only the managed setup changes documented by the pinned protocol, require my explicit
approval for model-alias changes, then report the resolved SHA and observed result.
```

Copie o bloco inteiro. Se o seu leitor de PDF bagunçar a cópia, o mesmo pedido está no README, logo no começo (Quick Start).

• PASSO 2

2 INSTALE O COMANDO OMH

Escolha um caminho. Todos deixam só o comando `omh` na máquina; nenhum roda o setup por você.

macOS e Linux

```
curl -fsSL https://raw.githubusercontent.com/rlaope/oh-my-hermes/main/install.sh | sh
```

Windows (PowerShell 5.1+)

```
irm https://raw.githubusercontent.com/rlaope/oh-my-hermes/main/install.ps1 | iex
```

Gerenciadores de pacote

```
brew install rlaope/tap/omh
bun install -g oh-my-hermes
npm install -g oh-my-hermes
```

A documentação de instalação marca o Bun como recomendado. Qualquer caminho instala o mesmo comando.

Versão fixa

Pelo curl ou pelo PowerShell, o instalador pega a versão lançada mais recente (canal `stable`). Para instalar exatamente a versão deste guia, diga qual:

```
curl -fsSL https://raw.githubusercontent.com/rlaope/oh-my-hermes/main/install.sh |
  OMH_CHANNEL=stable OMH_VERSION=4.0.0 sh
```

```
$env:OMH_VERSION = "4.0.0"
irm https://raw.githubusercontent.com/rlaope/oh-my-hermes/main/install.ps1 | iex
```

O protocolo para agentes (`INSTALL_FOR_AGENTS.md`) faz outra coisa: fixa um commit do `main`, que é código ainda não lançado (canal `preview`). Esse download é o repositório inteiro, cerca de 46 MB, e já levou mais de cinco minutos. Use esse caminho só se precisar de algo que ainda não saiu numa versão.

PASSO 3

3 CONFIGURE E CONFIRA

A instalação só prepara o comando. É o setup que liga o OMH ao Hermes; o doctor confere se ficou tudo certo.

```
omh setup
omh doctor
```

O que o `omh setup` faz

- Instala as skills em `~/.omh/skills`. Por padrão vem o perfil completo; `--core` instala a versão leve.
- Registra essa pasta no config do Hermes e instala a ponte do plugin em `~/.hermes/plugins/omh`.
- Ativa o plugin e a memória do OMH só se o espaço estiver livre. O que já é de outro produto fica como está.
- Faz o mesmo registro em cada perfil de bot do Hermes.
- Não mexe no núcleo do Hermes: `hermes` continua o Hermes original e `omh` abre a tela do OMH.

O que olhar no `omh doctor`

`ok`	Resultado geral e a próxima ação recomendada.
`blocking`	Item que precisa de correção antes de seguir.
`warning`	Aviso. Leia, mas não trava.
`command_path`	Se o `omh` está no PATH ou só no caminho absoluto.
`omh_tui_support`	Se a tela do OMH abre nesta máquina.
`hermes_config`	Qual config.yaml foi conferido.

Rodando o setup no terminal, ele pergunta quais provedores você usa (os que o Hermes já tem ligados vêm marcados) e, no fim, oferece revisar as cadeias de modelo.

- **Reinicie o Hermes** (ou o Hermes Desktop) para ele carregar as skills. Depois, mande o primeiro teste oficial. Se o Hermes já carregou o OMH, ele explica por que esse é o fluxo certo e diz qual papel cuida do pedido (como `planner`):

```
Use OMH request-to-handoff for: I want to safely add a feature to this repo.
```

- `omh` não encontrado? Use o caminho absoluto que o instalador imprimiu ou ponha a pasta no PATH. É aviso de comando, não sinal de que o registro falhou.

• PASSO 4

4 MODELO POR CATEGORIA

Cada categoria é uma cadeia de modelo + esforço que você edita sem tocar em código.

```
omh model
```

↑ ↓ categoria | ← → modelo principal | - / + esforço | d volta ao padrão | Enter salva | Esc sai sem salvar

CATEGORIA	PARA QUÊ	ORDEM RECOMENDADA (ESFORÇO)
ultrabrain	Raciocínio mais profundo	GPT-6 Astra xhigh
deep	Nível forte padrão	Claude Opus 5.5 > GPT-6.1 Sol > Kimi K3 high
architect	Arquitetura e sistemas	Claude Opus 5.5 > GPT-6 Astra > Claude Fable 5.1 > Kimi K3 xhigh
unspecified-high	Modelo de trabalho padrão	GPT-6.1 Sol > Claude Opus 5.5 > Kimi K3 medium
unspecified-low	Alternativa mais barata	GLM 5.3 > DeepSeek V4.1 Flash > GPT-6.1 Sol > Claude Sonnet 5.5 low
quick	Tarefas curtas	GLM 5.3 Flash > GPT-6 Luna > Claude Haiku 5.5 > Kimi K3 low
writing	Texto e documentação	Claude Opus 5.5 > Claude Sonnet 5.5 > Kimi K3 > Gemini 3.1 Pro medium
visual-engineering	Frontend e visual	Claude Opus 5.5 > Claude Fable 5.1 > Gemini 3.1 Pro > Kimi K3 high
artistry	Fora do convencional	Gemini 3.1 Pro > Claude Opus 5.5 > Claude Fable 5.1 > Kimi K3 high
capable	Trabalho geral forte	Claude Opus 5.5 > GPT-6.1 Sol > Claude Fable 5.1 > Kimi K3 medium
simple-work	Pequenas, do dia a dia	GPT-6 Luna > Claude Haiku 5.5 > DeepSeek V4.1 Flash low
deep-work	Longas, no nível de ponta	Claude Opus 5.5 > GPT-6.1 Sol max

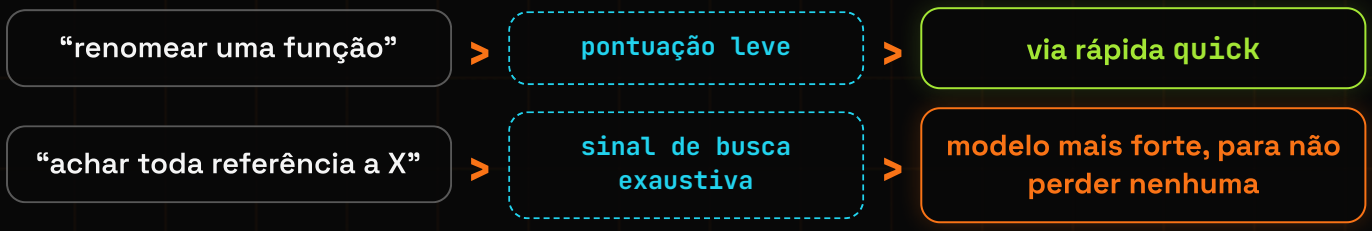
Essas cadeias são preferências editáveis do autor, não resultado de benchmark. Dentro da tela do OMH, o mesmo seletor abre com `/omh-model`. Para incluir uma família nova de modelos, use a skill `/omh-model-setup` no Hermes.

Tudo fica num arquivo só: `~/.omh/routing/model-chains.json`. Com `categories` vazio, valem as cadeias de fábrica, mas reordenadas pelos provedores que o seu Hermes tem ligados (o OMH lê só os nomes, nunca chave ou token), e elas podem mudar a cada `omh update`. A ordem real desta máquina aparece em `omh model-chains show`.

• COMO ELE DECIDE

ROTEAMENTO E TELA AO VIVO

Todo pedido recebe uma pontuação antes de sair, e cada sinal que mexeu nela aparece com nome.



É a intenção, não garantia: nos testes do autor, só os modelos GPT acharam todas (6 de 6); o GLM achou 2 e o Kimi, nenhuma. Conta qual modelo lidera a cadeia `unspecified-high`.

Quando um modelo falha

- O provedor recusou o modelo? A cadeia passa para o próximo da lista.
- Um despacho que herdaria um provedor incapaz de servir aquele modelo é **recusado**, e não rebaixado em silêncio.

A tela do OMH

Digite `omh`, sem mais nada, e a tela do OMH abre sobre o seu Hermes, com a lista de fases da execução acima do prompt. Cada tarefa delegada ganha uma linha:

`categoria:nome(modelo:esforço)`	`turno`	`tokens`	`custo`	`estado`
por onde foi	em que volta está	quanto consumiu	quanto custou	o que foi provado

Estados honestos

`Plan · not run`	Plano ou prompt pronto. **Nada rodou ainda.**
`Code · running`	Um executor está rodando agora, e o OMH acompanha.
`Code · reported done`	O executor disse que terminou. Ninguém conferiu.
`Test · verified`	Um teste, revisão ou portão de CI passou de verdade.

Custo sem maquiagem. Sem preço, a linha mostra `$0.0000 (unknown)`: o zero vem marcado como desconhecido e não quer dizer grátis. Zero sem marca só aparece quando o host confirmou o valor. Estimativa, da tabela do OMH ou de um preço que você mesmo gravou, vem com `~`. Tarefa filha que não fez nada aparece como `failed`, nunca como linha verde.

• DENTRO DA EXECUÇÃO

SKILLS E MEMÓRIA REVISADA

108 skills de especialista `omh-*`. Você nunca chama uma delas.

Quando o pedido toca uma dessas áreas, a skill certa já entra na execução como chamada de ferramenta e sobe a régua do que o agente aceita como pronto.

`frontend` `backend` `Rust` `depuração nativa` `serviço de inferência` `qualidade de design` `portões de verificação` `revisão de segurança` `orçamento de performance` `planos de refatoração` `e mais`

O README diz que o roteador entende pedidos em inglês ou coreano. Se em português a skill esperada não entrar, repita o pedido em inglês.

Fluxos para chamar pelo nome

`ulw-interview`	Pergunta uma coisa de cada vez até saber exatamente o que você quer.
`ulw-plan`	Plano revisado, com opções comparadas, riscos e critério de pronto.
`ulw-work`	Roda o plano aceito em frentes paralelas que não dividem arquivo.
`ulw-loop`	Repete planejar, construir e revisar até o objetivo passar de verdade.

```
ulw-loop: keep going until npm test passes
```

Memória de longo prazo, com revisão

`SESSÃO` › `CARTÃO DE REVISÃO` › `LEMBRADA` `RECUSADA` `ADIADA`

- **Quem revisa?** No modo padrão (`auto-safe`), o próprio OMH aprova na hora o que é seguro e retém ou recusa o que é sensível, como credenciais e logs. Você não precisa aprovar nada. Cada decisão fica com o motivo escrito. Para aprovar cada lembrança você mesmo: `omh setup --memory-mode review-first` e depois `omh memory approve`.
- Registro aprovado guarda a origem e uma data de revisão. Confirmar zera o relógio; o silêncio envelhece a memória de ativa para referência e depois para arquivo.
- A sessão seguinte recebe um pacote de lembranças ordenado para a tarefa e cortado num orçamento de tokens, sem conflitos nem duplicatas.
- A memória do próprio Hermes não é lida nem alterada. Esta é do OMH, em arquivo e revisada.

• PROVA REAL

MEÇA NA SUA CONTA

TESTE DO AUTOR, NO README · ASTRA SOZINHO × ASTRA + OMH

~~US$ 4,29~~
US$ 0,66
CUSTO ESTIMADO

~~23 MIN~~
5 MIN
TEMPO TOTAL

18 DE 30
RESOLVIDAS NOS DOIS CASOS

Mesmas 30 tarefas de código do autor. Sem OMH, o GPT-6 Astra (xhigh) fez todas. Com OMH, o roteador mandou as leves para um modelo barato (GLM 5.2 Ultrafast) e só as de busca exaustiva ficaram no Astra. **A economia vem dessa troca de modelo.** O lado com OMH foi montado com rodadas separadas de cada modelo, e o custo vem da tabela de preços do OMH, não da fatura.

A única comparação Hermes sozinho × Hermes com OMH que o README descreve (`benchmarks/product-ab/v1`) ainda não tem execução publicada.

Um teste honesto em 6 passos

1. **Escolha de 3 a 5 tarefas reais** que você repete (renomear algo, achar toda referência a uma função, fazer um teste passar). Use o mesmo texto de pedido nas duas rodadas.
2. **Congele o ponto de partida.** Anote o commit e volte para ele antes de cada tarefa.
3. **Rodada A, antes de instalar o OMH.** Use o Hermes com o seu modelo principal. Anote início, fim e o gasto no painel do provedor antes e depois de cada tarefa.
4. **Rodada B, com o OMH.** Instale, rode o setup e repita as tarefas: mesmo commit, mesmo modelo principal. O OMH pode mandar parte delas a outros modelos; é o que você mede.
5. **Compare os acertos primeiro.** Conte quantas tarefas cada rodada resolveu. O autor somou custo e tempo das 30 tarefas, e os dois lados acertaram o mesmo número. Se os seus acertos forem diferentes, compare só as tarefas resolvidas nos dois lados.
6. **Use o HUD para entender, os painéis para somar.** O HUD mostra tokens e custo por linha. O gasto, some pelos painéis dos provedores: mesma régua nas duas rodadas.

TAREFA	RODADA A · SEM OMH			RODADA B · COM OMH		
	TEMPO	US$	PASSOU?	TEMPO	US$	PASSOU?
1						
2						
3						
4						
5						

• MANUTENÇÃO

ATUALIZAR E DESINSTALAR

Atualizar

```
omh update
omh doctor
```

O `omh update` descobre como você instalou (Homebrew, Bun, npm, curl ou PowerShell), atualiza pelo mesmo caminho e renova skills, plugin e registro no Hermes. Depois, reinicie o Hermes.

Instalou por curl ou PowerShell? O `omh update` simples puxa o `main`, ainda não lançado. Para ficar numa versão lançada:

```
omh update --channel stable \
  --version 4.0.0
```

Desinstalar

```
omh uninstall --dry-run
omh uninstall
```

`--dry-run` mostra o que sairia, sem gravar nada. O `omh uninstall` tira o registro do Hermes, apaga o `~/.omh` e a ponte do plugin, limpa os perfis de bot e só desfaz uma chave do config se ela ainda tiver o valor que o OMH gravou.

Instalou por Homebrew, Bun ou npm? Rode `omh uninstall --all` e depois o remover do gerenciador. Tirar só o pacote preserva o estado, inclusive a memória revisada.

> Nunca rode o instalador curl por cima de uma instalação por gerenciador de pacote: isso cria um segundo `omh`.

Onde conferir

`github.com/rlaope/oh-my-hermes` `README.md` `docs/INSTALLATION.md` `INSTALL_FOR_AGENTS.md` `rlaope.github.io/oh-my-hermes`

HERMES WEEK

Toda semana, a Hermes Week ensina o Hermes no detalhe, do zero.

`@redpro.ia`

Projeto: github.com/rlaope/oh-my-hermes (MIT). Guia: @redpro.ia